

ZENTRAW 3D VISUALIZER - CÓDIGO COMPLETO E ESTRUTURAS V1.4.0.a.2

****Última Atualização:**** 21 de Julho de 2025

****Status Atual:**** Sistema em Desenvolvimento - Backend Configurado, Problema de Execução Blender

****Branch Atual:**** feat_V1.4.0.a.2_BRIGA_COM_SERVIDOR_E_BACKEND

📊 RESUMO EXECUTIVO ATUALIZADO

🎯 STATUS ATUAL (21/07/2025):

- ✅ ****Backend Structure****: Implementado com CORS e Static File Serving
- ✅ ****Image Proxy****: Resolvido via express.static middleware
- ✅ ****Frontend Interface****: Interface Specterr-style completa (834 linhas)
- ✅ ****Sistema Robusto****: BlenderServiceRobust com 5 métodos de fallback
- ❌ ****Blender Execution****: Problemas críticos com spawn/execFile no Windows
- ❌ ****End-to-End Flow****: Não funcional devido a problemas de execução

🚩 PROBLEMAS CRÍTICOS IDENTIFICADOS:

1. ****Windows Path Handling****: Blender em "C:\Program Files\Blender Foundation\Blender 4.5\blender.exe"
2. ****Process Spawn Failures****: Exit code 1, timeout em spawn
3. ****Backend Connectivity****: Problemas intermitentes de inicialização
4. ****Terminal Hanging****: Comandos não respondem adequadamente

📁 ESTRUTURA DE ARQUIVOS COMPLETA (ATUALIZADA)

1. SERVER BACKEND (IMPLEMENTAÇÃO ATUAL)

`backend-only.ts` - Express Server Principal (VERSÃO ATUALIZADA)

```
``typescript
```

```
import express from "express";
```

```
import path from "path";
```

```
import { registerRoutes } from "./routes.js";
```

```
import blenderRouter from "./routes/blender.js";
```

```
const app = express();
```

```
// CORS middleware simples
```

```
app.use((req, res, next) => {
```

```
  const allowedOrigins = ['http://localhost:5173', 'http://localhost:5174',
```

```
  'http://localhost:5175', 'http://localhost:5176'];
```

```
  const origin = req.headers.origin;
```

```
  if (origin && allowedOrigins.includes(origin)) {
```

```
    res.header('Access-Control-Allow-Origin', origin);
```

```
  }
```

```
  res.header('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
```

```
  res.header('Access-Control-Allow-Headers', 'Content-Type, Authorization');
```

```
  res.header('Access-Control-Allow-Credentials', 'true');
```

```
  if (req.method === 'OPTIONS') {
```

```
    res.sendStatus(200);
```

```
    } else {  
      next();  
    }  
  });  
});
```

```
// ✅ SOLUÇÃO: Servir arquivos estáticos da pasta uploads  
const uploadsPath = path.join(process.cwd(), 'uploads');  
app.use('/uploads', express.static(uploadsPath));  
console.log('📁 Static files serving from:', uploadsPath);
```

```
app.use(express.json({ limit: '50mb' }));  
app.use(express.urlencoded({ extended: true, limit: '50mb' }));
```

```
// Log middleware  
app.use((req, res, next) => {  
  const timestamp = new Date().toLocaleTimeString();  
  console.log(`[${timestamp} [backend] ${req.method} ${req.path}]`);  
  next();  
});
```

```
// ✅ ADICIONANDO ROTAS NECESSÁRIAS  
// Health check  
app.get('/health', (req, res) => {  
  res.json({  
    status: 'ok',  
    timestamp: new Date().toISOString(),  
    message: 'Zentraw 3D Visualizer Backend is running!',  
    version: '1.4.0.a.2'  
  });  
});
```

```
// Root endpoint  
app.get('/', (req, res) => {  
  res.json({  
    message: 'Zentraw 3D Visualizer Backend V1.4.0.a.2',  
    status: 'running',  
    endpoints: {  
      health: '/health',  
      blenderTest: '/api/blender/test',  
      blenderPreview: '/api/blender/preview'  
    }  
  });  
});
```

```
// Blender routes  
app.use('/api/blender', blenderRouter);
```

```
// Outras rotas (se necessário)  
registerRoutes(app);
```

```
// Error handler
```

```

app.use((err: any, _req: any, res: any, _next: any) => {
  const status = err.status || err.statusCode || 500;
  const message = err.message || "Internal Server Error";
  res.status(status).json({ message });
  console.error(err);
});

// Adicionando logs detalhados para inicialização
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`🚀 Backend iniciado com sucesso na porta ${PORT}`);
}).on('error', (err) => {
  console.error('❌ Erro ao iniciar o backend:', err);
});

```

`routes/blender.ts` - API Routes (IMPLEMENTAÇÃO ATUAL)

```

``typescript
import express, { Request, Response } from 'express';
import multer, { FileFilterCallback } from 'multer';
import path from 'path';
import fs from 'fs';
import { BlenderService } from '../services/blender-service.js';

const router = express.Router();

// Configurar Multer para upload de arquivos
const storage = multer.diskStorage({
  destination: (req: Request, file: Express.Multer.File, cb: (error: Error | null, destination: string) => void) => {
    const uploadDir = path.join(process.cwd(), 'uploads', 'blender');
    if (!fs.existsSync(uploadDir)) {
      fs.mkdirSync(uploadDir, { recursive: true });
    }
    cb(null, uploadDir);
  },
  filename: (req: Request, file: Express.Multer.File, cb: (error: Error | null, filename: string) => void) => {
    const timestamp = Date.now();
    const ext = path.extname(file.originalname);
    const name = path.basename(file.originalname, ext);
    cb(null, `${name}_${timestamp}${ext}`);
  }
});

const upload = multer({
  storage,
  limits: {
    fileSize: 100 * 1024 * 1024 // 100MB limite
  },
  fileFilter: (req: Request, file: Express.Multer.File, cb: FileFilterCallback) => {

```

```

if (file.fieldname === 'audio') {
  // Aceitar arquivos de audio
  if (file.mimetype.startsWith('audio/')) {
    cb(null, true);
  } else {
    cb(new Error('Only audio files are allowed for audio field'));
  }
} else if (file.fieldname === 'image') {
  // Aceitar arquivos de imagem
  if (file.mimetype.startsWith('image/')) {
    cb(null, true);
  } else {
    cb(new Error('Only image files are allowed for image field'));
  }
} else {
  cb(new Error('Unexpected field'));
}
}
});

/**
 * GET /api/blender/test
 * Testa se o Blender está funcionando
 */
router.get('/test', async (req: Request, res: Response) => {
  try {
    console.log('🔧 Testing Blender installation...');
    const result = await BlenderService.testBlenderInstallation();
    res.json({
      success: true,
      blenderAvailable: result,
      message: result ? 'Blender is working correctly' : 'Blender test failed',
      timestamp: new Date().toISOString(),
      version: '1.4.0.a.2'
    });
  } catch (error) {
    console.error('❌ Blender test error:', error);
    res.status(500).json({
      success: false,
      error: error instanceof Error ? error.message : 'Unknown error',
      timestamp: new Date().toISOString()
    });
  }
});

/**
 * POST /api/blender/preview
 * Gera um preview 3D
 */
router.post('/preview', upload.fields([
  { name: 'audio', maxCount: 1 },

```

```

    { name: 'image', maxCount: 1 }
  )), async (req: Request, res: Response) => {
    try {
      const files = req.files as { [fieldname: string]: Express.Multer.File[] };
      const audioFile = files.audio?.[0];
      const imageFile = files.image?.[0];

      if (!audioFile) {
        return res.status(400).json({
          success: false,
          error: 'Audio file is required'
        });
      }

      console.log('🎵 Received preview request:', {
        audio: audioFile.originalname,
        image: imageFile ? imageFile.originalname : 'none',
        audioSize: audioFile.size,
        imageSize: imageFile ? imageFile.size : 0
      });

      // Gerar preview usando BlenderService
      const result = await BlenderService.generatePreview({
        audioFile: audioFile.path,
        imageFile: imageFile?.path || '',
        renderEngine: 'eevee',
        cameraSettings: {
          distance: parseFloat(req.body.cameraDistance) || 10,
          height: parseFloat(req.body.cameraHeight) || 5,
          angle: parseFloat(req.body.cameraAngle) || 0
        },
        animationStyle: req.body.animationStyle || 'cube',
        sensitivity: parseFloat(req.body.sensitivity) || 1.0,
        smoothing: parseFloat(req.body.smoothing) || 0.5
      });

      if (result.success && result.previewPath) {
        // Retornar URL relativa para static file serving
        const filename = path.basename(result.previewPath);
        res.json({
          success: true,
          previewUrl: `/uploads/blender/${filename}`,
          renderTime: result.renderTime,
          message: 'Preview generated successfully'
        });
      } else {
        res.status(500).json({
          success: false,
          error: result.error || 'Failed to generate preview'
        });
      }
    }
  }

```

```

    } catch (error) {
      console.error('❌ Preview generation error:', error);
      res.status(500).json({
        success: false,
        error: error instanceof Error ? error.message : 'Unknown error'
      });
    }
  });
});

export default router;
```

```

## ### 2. SERVICES LAYER (SISTEMA ROBUSTO IMPLEMENTADO)

#### `services/blender-service.ts` - Serviço Principal (VERSÃO ATUAL)

```

``typescript
import { spawn, execFile, execSync } from 'child_process';
import path from 'path';
import fs from 'fs';
import { BLENDER_PATHS } from '../blender-paths.js';
import { BlenderServiceRobust } from './blender-service-robust.js';

```

```

export interface BlenderRenderOptions {
 audioPath: string;
 imagePath: string;
 templatePath?: string;
 outputPath?: string;
}

```

```

export interface BlenderRenderResult {
 success: boolean;
 outputPath?: string;
 error?: string;
 duration?: number;
}

```

```

export interface PreviewOptions {
 audioFile: string;
 imageFile: string;
 renderEngine: 'eevee' | 'cycles';
 cameraSettings: {
 distance: number;
 height: number;
 angle: number;
 };
 animationStyle: 'cube' | 'sphere' | 'bars';
 sensitivity: number;
 smoothing: number;
}

```

```

export interface PreviewResult {
 success: boolean;
}

```

```
previewPath?: string;
error?: string;
renderTime?: number;
}
```

```
export class BlenderService {
 private static readonly BLENDER_PATH = BLENDER_PATHS.BLENDER_EXE;
 private static readonly SCRIPT_PATH = BLENDER_PATHS.SCRIPT_PATH;
 private static readonly DEFAULT_TEMPLATE = BLENDER_PATHS.TEMPLATE_PATH;
```

```
// Log para debug - forçar reload
static {
 console.log('🔄 BlenderService reloaded - Template path:',
BlenderService.DEFAULT_TEMPLATE);
}
```

```
/**
 * Testa se o Blender está instalado e funcionando
 */
static async testBlenderInstallation(): Promise<boolean> {
 try {
 console.log('🔧 Testing Blender at:', this.BLENDER_PATH);

 if (!fs.existsSync(this.BLENDER_PATH)) {
 console.error('❌ Blender executable not found');
 return false;
 }
 }
```

```
// Usar BlenderServiceRobust para teste robusto
return await BlenderServiceRobust.testBlenderInstallation();

} catch (error) {
 console.error('❌ Blender test failed:', error);
 return false;
}
}
```

```
/**
 * Gera um preview 3D usando o sistema robusto
 */
static async generatePreview(options: PreviewOptions): Promise<PreviewResult> {
 try {
 console.log('🖼️ Generating preview with robust system...');

 // Delegar para BlenderServiceRobust
 return await BlenderServiceRobust.generatePreviewWithFallback(options);

 } catch (error) {
 console.error('❌ Preview generation failed:', error);
 return {
 success: false,
```

```

 error: error instanceof Error ? error.message : 'Unknown error'
 };
}
}

/**
 * Renderiza um audio visualizer usando Blender (Método Principal)
 */
static async renderAudioVisualizer(options: BlenderRenderOptions):
Promise<BlenderRenderResult> {
 const startTime = Date.now();
 const outputPath = options.outputPath || path.join(process.cwd(), 'Blender', 'output.mp4');
 const templatePath = options.templatePath || this.DEFAULT_TEMPLATE;

 try {
 // Validar se os arquivos existem
 if (!fs.existsSync(options.audioPath)) {
 throw new Error(`Audio file not found: ${options.audioPath}`);
 }

 if (!fs.existsSync(options.imagePath)) {
 throw new Error(`Image file not found: ${options.imagePath}`);
 }

 if (!fs.existsSync(templatePath)) {
 throw new Error(`Template file not found: ${templatePath}`);
 }

 if (!fs.existsSync(this.BLENDER_PATH)) {
 throw new Error(`Blender not found at: ${this.BLENDER_PATH}`);
 }

 // Usar sistema robusto para execução
 const result = await BlenderServiceRobust.executeBlenderRender({
 blenderPath: this.BLENDER_PATH,
 templatePath,
 audioPath: options.audioPath,
 imagePath: options.imagePath,
 outputPath
 });

 return {
 success: result.success,
 outputPath: result.success ? outputPath : undefined,
 error: result.error,
 duration: Date.now() - startTime
 };
 } catch (error) {
 console.error('❌ Render failed:', error);
 return {
 success: false,

```



```

 error: error instanceof Error ? error.message : 'Unknown error',
 duration: Date.now() - startTime
 };
 }
}
}
...

```

#### `routes/blender.ts` - API Routes

```
``typescript
```

```

import express from 'express';
import { BlenderService } from '../services/blender-service.js';
import multer from 'multer';
import path from 'path';

```

```

const router = express.Router();
const upload = multer({ dest: 'uploads/' });

```

```
// Test endpoint
```

```

router.get('/test', async (req, res) => {
 try {
 const isWorking = await BlenderService.testBlenderInstallation();
 res.json({
 success: isWorking,
 message: isWorking ? 'Blender is working' : 'Blender test failed'
 });
 } catch (error) {
 res.status(500).json({
 success: false,
 error: error instanceof Error ? error.message : 'Unknown error'
 });
 }
});

```

```
// Preview generation endpoint
```

```

router.post('/preview', upload.fields([
 { name: 'audioFile', maxCount: 1 },
 { name: 'imageFile', maxCount: 1 }
]), async (req, res) => {
 try {
 const files = req.files as { [fieldname: string]: Express.Multer.File[] };
 const audioFile = files.audioFile?.[0];
 const imageFile = files.imageFile?.[0];

 if (!audioFile || !imageFile) {
 return res.status(400).json({
 success: false,
 error: 'Audio and image files are required'
 });
 }
 }
}

```

```
const options = {
```

```

 audioFile: audioFile.path,
 imageFile: imageFile.path,
 renderEngine: (req.body.renderEngine || 'eevee') as 'eevee' | 'cycles',
 cameraSettings: {
 distance: parseInt(req.body.distance) || 50,
 height: parseInt(req.body.height) || 50,
 angle: parseInt(req.body.angle) || 0
 },
 animationStyle: (req.body.animationStyle || 'cube') as 'cube' | 'sphere' | 'bars',
 sensitivity: parseInt(req.body.sensitivity) || 50,
 smoothing: parseInt(req.body.smoothing) || 30
 };

const blenderService = new BlenderService();
const result = await blenderService.generatePreview(options);

if (result.success && result.previewPath) {
 const filename = path.basename(result.previewPath);

 // ✅ IMAGE PROXY - Return URL for static serving
 const previewUrl = `/uploads/blender/${filename}`;

 res.json({
 success: true,
 previewUrl,
 renderTime: result.renderTime,
 message: 'Preview generated successfully'
 });
} else {
 res.status(500).json({
 success: false,
 error: result.error || 'Failed to generate preview'
 });
}
} catch (error) {
 console.error('❌ Preview generation error:', error);
 res.status(500).json({
 success: false,
 error: error instanceof Error ? error.message : 'Unknown error'
 });
}
});

export { router as blenderRouter };
```

```

2. BLENDER SERVICE - PROBLEMA CRÍTICO

```

##### `services/blender-service.ts` - Serviço Principal (ATUAL COM PROBLEMAS)
```typescript
import { spawn, execFile } from 'child_process';

```

```

import path from 'path';
import fs from 'fs';
import { BLENDER_PATHS } from '../blender-paths.js';

export class BlenderService {
 private static readonly BLENDER_PATH = BLENDER_PATHS.BLENDER_EXE;
 private static readonly SCRIPT_PATH = BLENDER_PATHS.SCRIPT_PATH;
 private static readonly DEFAULT_TEMPLATE = BLENDER_PATHS.TEMPLATE_PATH;

 // ❌ PROBLEMA: Este método falha na execução do Blender
 private static executeBlender(templatePath: string, scriptArgs: string[]): Promise<{ success:
boolean; error?: string }> {
 return new Promise((resolve) => {
 const args = [
 '--background',
 templatePath,
 '--python',
 this.SCRIPT_PATH,
 '--',
 ...scriptArgs
];

 console.log(`Executing: ${this.BLENDER_PATH} ${args.join(' ')}`);

 // ❌ PROBLEMA: spawn falha com paths que têm espaços
 const blenderProcess = spawn(this.BLENDER_PATH, args, {
 stdio: ['pipe', 'pipe', 'pipe'],
 shell: true // Tentativa atual - não resolve
 });

 // Timeout e error handling
 let hasError = false;
 const timeout = setTimeout(() => {
 console.log('❌ Blender process timeout, killing...');
 blenderProcess.kill('SIGTERM');
 hasError = true;
 }, 30000);

 blenderProcess.on('error', (error) => {
 console.error('❌ Blender spawn error:', error.message);
 clearTimeout(timeout);
 hasError = true;
 resolve({
 success: false,
 error: `Failed to start Blender: ${error.message}`
 });
 });

 blenderProcess.on('close', (code) => {
 clearTimeout(timeout);
 if (hasError) return;

```

```

 if (code === 0) {
 console.log('✅ Blender process completed successfully');
 resolve({ success: true });
 } else {
 console.error(`❌ Blender process exited with code ${code}`);
 resolve({
 success: false,
 error: `Blender process failed with exit code ${code}`
 });
 }
 });
});
}

```

```

// ❌ PROBLEMA: generatePreview usa execFile que também falha
async generatePreview(options: PreviewOptions): Promise<PreviewResult> {
 const startTime = Date.now();
 const timestamp = Date.now();
 const uploadsDir = path.join(process.cwd(), 'uploads', 'blender');

 if (!fs.existsSync(uploadsDir)) {
 fs.mkdirSync(uploadsDir, { recursive: true });
 }

```

```

 const outputPath = path.join(uploadsDir, `preview_${timestamp}.png`);
 const templatePath = path.join(process.cwd(), 'Blender', 'template.blend');

```

```

 return new Promise((resolve) => {
 const args = [
 '--background',
 templatePath,
 '--python',
 path.join(process.cwd(), 'Blender', 'preview_script.py'),
 '--',
 outputPath
];

```

```

// ❌ PROBLEMA: execFile também falha com paths com espaços
const blenderProcess = execFile(BlenderService.BLENDER_PATH, args, {
 maxBuffer: 1024 * 1024 * 10,
 timeout: 30000
}, (error, stdout, stderr) => {
 const renderTime = Date.now() - startTime;

```

```

 if (error) {
 console.error(`❌ Blender execFile error:`, error);
 resolve({
 success: false,
 error: `Blender execution failed: ${error.message}`,
 renderTime

```

```

 });
 return;
 }

 if (fs.existsSync(outputPath)) {
 resolve({
 success: true,
 previewPath: outputPath,
 renderTime
 });
 } else {
 resolve({
 success: false,
 error: `Preview file was not generated. Stderr: ${stderr}`,
 renderTime
 });
 }
 });
});
}
}
...

```

### ### 3. FRONTEND CLIENT

#### `client/components/blender-visualizer.tsx` - Interface React

```typescript

```
import React, { useState, useRef } from 'react';
```

```
interface BlenderVisualizerProps {
  // Props interface
}
```

```
export const BlenderVisualizer: React.FC<BlenderVisualizerProps> = () => {
  const [isGenerating, setIsGenerating] = useState(false);
  const [previewUrl, setPreviewUrl] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
```

```
  const generatePreview = async () => {
    setIsGenerating(true);
    setError(null);
```

```
    try {
      const formData = new FormData();
      // Adicionar arquivos e configurações
```

```
      const response = await fetch('/api/blender/preview', {
        method: 'POST',
        body: formData
      });
```

```
      const result = await response.json();
```

```

    if (result.success) {
      //  IMAGE PROXY - Usar URL completa para acessar imagem
      const fullUrl = `http://localhost:5001${result.previewUrl}`;
      setPreviewUrl(fullUrl);
    } else {
      setError(result.error || 'Failed to generate preview');
    }
  } catch (err) {
    setError('Network error occurred');
  } finally {
    setIsGenerating(false);
  }
};

return (
  <div className="blender-visualizer">
    <button
      onClick={generatePreview}
      disabled={isGenerating}
      className="generate-btn"
    >
      {isGenerating ? 'Generating...' : 'Generate Preview'}
    </button>

    {previewUrl && (
      <div className="preview-container">
        <img src={previewUrl} alt="3D Preview" />
      </div>
    )}

    {error && (
      <div className="error-message">
        {error}
      </div>
    )}
  </div>
);
};
...

```

4. BLENDER INTEGRATION

`Blender/preview\_script.py` - Script Python para Preview

```

'''python
import bpy
import sys
import os

def render_preview(output_path):
    """Renderiza um frame de preview"""
    try:

```

```

# Configurar render settings
bpy.context.scene.render.engine = 'EVEE'
bpy.context.scene.render.resolution_x = 800
bpy.context.scene.render.resolution_y = 600
bpy.context.scene.render.filepath = output_path

# Renderizar
bpy.ops.render.render(write_still=True)
print(f"Preview rendered: {output_path}")

except Exception as e:
    print(f"Error rendering preview: {str(e)}")
    sys.exit(1)

if __name__ == "__main__":
    if len(sys.argv) > 1:
        output_path = sys.argv[1]
        render_preview(output_path)
    else:
        print("Usage: blender --background template.blend --python preview_script.py --
output.png")
        sys.exit(1)
...

#### `Blender/template.blend` - Arquivo 3D Template
...

[ARQUIVO BINÁRIO BLENDER]
- Contém setup 3D para visualização
- Configurações de câmera, luzes, materiais
- Objetos 3D preparados para audio visualization
- Funcionou perfeitamente ontem (16/07/2025)
...

## 🔍 ANÁLISE DE PROBLEMAS

### 1. WINDOWS PATH ISSUE - ANÁLISE DETALHADA
```typescript
// ❌ PROBLEMA: Path com espaços não funciona
const BLENDER_PATH = 'C:\\Program Files\\Blender Foundation\\Blender 4.5\\blender.exe';

// Tentativas que falharam:
spawn(BLENDER_PATH, args, { shell: false }); // Falha: comando não encontrado
spawn(BLENDER_PATH, args, { shell: true }); // Falha: exit code 1
execFile(BLENDER_PATH, args); // Falha: não inicia processo

// ✅ SOLUÇÕES POSSÍVEIS:
// 1. Usar path curto do Windows
const SHORT_PATH = 'C:\\PROGRA~1\\BLENDE~1\\BLENDE~1\\blender.exe';

// 2. Usar PowerShell
const powershell = spawn('powershell', ['-Command', '& "${BLENDER_PATH}" ${args.join(' ')}']);

```

```
// 3. Usar util.promisify com exec
const exec = util.promisify(require('child_process').exec);
const result = await exec(`${BLENDER_PATH} ${args.join(' ')}`);
...
```

### ### 2. PROCESS EXECUTION PATTERNS

```
``typescript
```

```
//  FUNCIONOU ONTEM (16/07/2025)
```

```
// Alguma configuração específica que permitiu execução
```

```
//  FALHA HOJE (17/07/2025)
```

```
// Mesma configuração não funciona mais
```

```
// POSSÍVEIS CAUSAS:
```

```
// - Windows Update
```

```
// - Node.js version change
```

```
// - Environment variables
```

```
// - Process permissions
```

```
// - Antivirus interference
```

```
...
```

### ### 3. IMAGE PROXY STATUS

```
``typescript
```

```
//  IMPLEMENTADO CORRETAMENTE
```

```
// Backend: express.static middleware
```

```
// Frontend: Full URL usage
```

```
// Routes: Correct URL generation
```

```
//  NÃO TESTÁVEL
```

```
// Depende de Blender execution working
```

```
// Não pode gerar arquivos para servir
```

```
...
```

## ## DEBUGGING CHECKLIST

### ### 1. VERIFICAÇÕES BÁSICAS

```
- [] Blender executable exists: `fs.existsSync(BLENDER_PATH)`
```

```
- [] Template file exists: `fs.existsSync(TEMPLATE_PATH)`
```

```
- [] Python script exists: `fs.existsSync(SCRIPT_PATH)`
```

```
- [] Permissions adequadas
```

```
- [] Antivirus não bloqueando
```

### ### 2. PROCESS DEBUGGING

```
``typescript
```

```
// Adicionar logs detalhados
```

```
console.log('PWD:', process.cwd());
```

```
console.log('PATH:', process.env.PATH);
```

```
console.log('Platform:', process.platform);
```

```
console.log('Node version:', process.version);
```



```
// Testar comando direto
const testCommand = `${BLENDER_PATH} --version`;
console.log('Test command:', testCommand);
````
```

3. ALTERNATIVE APPROACHES

```
````typescript
// 1. cross-spawn package
import spawn from 'cross-spawn';
const process = spawn(BLENDER_PATH, args);

// 2. shelljs
import shell from 'shelljs';
const result = shell.exec(`${BLENDER_PATH} ${args.join(' ')}`);

// 3. node-cmd
import cmd from 'node-cmd';
cmd.get(`${BLENDER_PATH} ${args.join(' ')}`, callback);
````
```

NEXT ACTIONS

1. IMMEDIATE FIXES NEEDED

1. ****Resolve Blender execution**** - Priority #1
2. ****Test alternative spawn methods**** - Priority #2
3. ****Validate image proxy end-to-end**** - Priority #3

2. IMPLEMENTATION PLAN

```
````typescript
// Step 1: Try cross-spawn
npm install cross-spawn
import spawn from 'cross-spawn';

// Step 2: Try short path
const BLENDER_SHORT = 'C:\\\\PROGRA~1\\\\BLENDE~1\\\\BLENDE~1\\\\blender.exe';

// Step 3: Try PowerShell wrapper
const powershellExec = (command: string) => {
 return spawn('powershell', ['-Command', command]);
};
````
```

3. SUCCESS CRITERIA

- [] Blender executes without errors
- [] Preview image generated successfully
- [] Image proxy serves file correctly
- [] Frontend displays preview
- [] 3D Visualizer V1.4.0.a.2 = 100% functional

****Status****: COMPREHENSIVE ANALYSIS COMPLETE

****Blocker****: Windows Blender execution via child_process

****Solution****: Implement alternative process execution method

****Timeline****: Critical - needs immediate resolution